System Software

Lab Assignment - 2

Name : Naishadh Rana

Enrollment Number : U23CS014

---

1. Write a C Program to recognize a valid arithmetic expression and identify the identifiers and operators present. Print them separately.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char identifiers[20][20];
char operators[20];
int id_count = 0, op_count = 0;

int main() {
   char expr[100];
   char token[20];
   int i, j, valid = 1;

   printf("Enter arithmetic expression: ");
   fgets(expr, sizeof(expr), stdin);
   expr[strlen(expr)-1] = '\0';  // remove newline

   for (i = 0; expr[i] != '\0'; i++) {
       // Skip spaces
       if (expr[i] == ' ' || expr[i] == '\n')
```

```
            continue;

        if (expr[i] == '+' || expr[i] == '-' || expr[i] == '*' ||
            expr[i] == '/' || expr[i] == '%') {
            operators[op_count] = expr[i];
            op_count++;
        }
        else if (isalpha(expr[i])) {
            j = 0;
            while (isalnum(expr[i])) {
                token[j] = expr[i];
                j++;
                i++;
            }
            token[j] = '\0';
            i--;
            strcpy(identifiers[id_count], token);
            id_count++;
        }
        else if (isdigit(expr[i])) {
            while (isdigit(expr[i]))
                i++;
            i--;
        }
        else if (expr[i] == '(' || expr[i] == ')') {
            continue;
        }
        else {
            valid = 0;
            break;
        }
    }

    if (valid) {
        printf("\nVALID arithmetic expression\n");

        printf("\nIdentifiers: ");
        for (i = 0; i < id_count; i++)
            printf("%s ", identifiers[i]);

        printf("\nOperators: ");
        for (i = 0; i < op_count; i++)
```

```
            printf("%c ", operators[i]);
        printf("\n");
    }
    else {
        printf("\nINVALID expression\n");
    }

    return 0;
}
```

Output:

```
Enter arithmetic expression: a+b-c*d
VALID arithmetic expression

Identifiers: a b c d
Operators: + - *
```

2. Write a C program to identify whether a given line is a comment or not.

```c
#include <stdio.h>
#include <string.h>

int main() {
   char line[100];
   int i = 0;

   printf("Enter a line: ");
   fgets(line, sizeof(line), stdin);

   while (line[i] == ' ' || line[i] == '\t') //skip leading spaces
       i++;

   printf("Result: ");

   if (line[i] == '/' && line[i+1] == '/') {
       printf("Single-line comment\n");
   }
   else if (line[i] == '/' && line[i+1] == '*') {
       if (strstr(line, "*/") != NULL)
           printf("Multi-line comment (complete)\n");
       else
           printf("Multi-line comment START\n");
   }
   else if (strstr(line, "*/") != NULL) {
       printf("Multi-line comment END\n");
   }
   else {
       printf("NOT a comment\n");
   }

   return 0;
}
```

Output:

```
naish@MacBook-Air Lab2 % ./comment_check
Enter a line: //hi
Result: Single-line comment
naish@MacBook-Air Lab2 % ./comment_check
Enter a line: /* comm. start
Result: Multi-line comment START
naish@MacBook-Air Lab2 % ./comment_check
Enter a line:  end of comment */
Result: Multi-line comment END
naish@MacBook-Air Lab2 % ./comment_check
Enter a line: /* complete comment */
Result: Multi-line comment (complete)
```

3. Create Assembler as discussed in Class.

Assembler.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "assembler.h"

// OPTAB - 8085 insts
struct OptabEntry OPTAB[] = {
   {"NOP",  0x00, 1, IK_FIXED},
   {"HLT",  0x76, 1, IK_FIXED},
   {"MVI",  0x00, 2, IK_MVI},
   {"LXI",  0x01, 3, IK_LXI},
   {"LDA",  0x3A, 3, IK_DIRECT},
   {"STA",  0x32, 3, IK_DIRECT},
   {"JMP",  0xC3, 3, IK_JUMP},
   {"JZ",   0xCA, 3, IK_JUMP},
   {"JNZ",  0xC2, 3, IK_JUMP},
   {"JC",   0xDA, 3, IK_JUMP},
   {"JNC",  0xD2, 3, IK_JUMP},
   {"MOV",  0x40, 1, IK_MOV},
   {"ADD",  0x80, 1, IK_REG},
   {"SUB",  0x90, 1, IK_REG},
   {"INR",  0x04, 1, IK_INRDCR},
   {"DCR",  0x05, 1, IK_INRDCR},
};

int OPTAB_SIZE = 16;
// optab search
struct OptabEntry* lookup_optab(char *mnemonic) {
   int i;
   for (i = 0; i < OPTAB_SIZE; i++) {
       if (strcasecmp(mnemonic, OPTAB[i].mnemonic) == 0)
           return &OPTAB[i];
   }
   return NULL;
}
```

```c
int main(int argc, char *argv[]) {
   struct Pass1Result p1 = {0};

   if (argc != 2) {
       printf("Usage: %s <source.asm>\n", argv[0]);
       return 1;
   }
   // Pass 1
   printf("Running Pass 1...\n");
   if (pass1(argv[1], "intermediate.int", &p1) != 0) {
       printf("Pass 1 failed\n");
       return 1;
   }
   printf("Pass 1 complete.\n");
   print_symtab(&p1.symtab);


   // Pass 2
   printf("\nRunning Pass 2...\n");
   if (pass2("intermediate.int", "output.obj", &p1) != 0) {
       printf("Pass 2 failed\n");
       return 1;
   }

   printf("\nAssembly complete!\n");
   printf("Output: intermediate.int, output.obj\n");

   return 0;
}
```

Custom header (assembler.h):

```c
#ifndef ASSEMBLER_H
#define ASSEMBLER_H
#define MAX_SYMBOLS 50

// Instruction types
#define IK_FIXED    0   // NOP, HLT
#define IK_MVI      1   // MVI r, data
#define IK_LXI      2   // LXI rp, data16
```

```c
#define IK_DIRECT   3   // LDA, STA addr
#define IK_JUMP     4   // JMP, JZ, JNZ addr
#define IK_MOV      5   // MOV r1, r2
#define IK_REG      6   // ADD, SUB r
#define IK_INRDCR   7   // INR, DCR r

struct OptabEntry {
   char mnemonic[10];
   int opcode;
   int length;
   int kind;
};
struct SymEntry {
   char label[20];
   int address;
};

struct Symtab {
   struct SymEntry entries[MAX_SYMBOLS];
   int count;
};

struct Pass1Result {
   struct Symtab symtab;
   int start_addr;
};

struct OptabEntry* lookup_optab(char *mnemonic);
int pass1(char *source, char *intermediate, struct Pass1Result *result);
int pass2(char *intermediate, char *output, struct Pass1Result *p1);
void print_symtab(struct Symtab *symtab);

#endif
```

Pass_1.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "assembler.h"
```

```c
// white space trimming, it caused errors in variables and mnemonic identification
void trim(char *s) {
   int i = 0, j = 0;
   while (s[i] == ' ' || s[i] == '\t') i++;
   while (s[i]) s[j++] = s[i++];
   s[j] = '\0';
   while (j > 0 && (s[j-1] == ' ' || s[j-1] == '\n' || s[j-1] == '\r'))
       s[--j] = '\0';
}

// decimal or hex with H suffix parse
int parse_number(char *s) {
   int len = strlen(s);
   if (len > 0 && (s[len-1] == 'H' || s[len-1] == 'h')) {
       s[len-1] = '\0';
       return (int)strtol(s, NULL, 16);
   }
   return atoi(s);
}

// directive check
int is_directive(char *op) {
   if (strcasecmp(op, "START") == 0) return 1;
   if (strcasecmp(op, "END") == 0) return 1;
   if (strcasecmp(op, "BYTE") == 0) return 1;
   if (strcasecmp(op, "WORD") == 0) return 1;
   if (strcasecmp(op, "RESB") == 0) return 1;
   if (strcasecmp(op, "RESW") == 0) return 1;
   return 0;
}

int add_symbol(struct Symtab *tab, char *label, int addr) {
   int i;
   for (i = 0; i < tab->count; i++) {
       if (strcmp(tab->entries[i].label, label) == 0) {
           printf("Error: Duplicate label %s\n", label);
           return -1;
       }
   }
   strcpy(tab->entries[tab->count].label, label);
   tab->entries[tab->count].address = addr;
```

```c
    tab->count++;
    return 0;
}

int pass1(char *source, char *intermediate, struct Pass1Result *result) {
    FILE *src, *out;
    char line[100], label[20], opcode[20], operand[50];
    char first[20], rest[80];
    int loc = 0;

    src = fopen(source, "r");
    out = fopen(intermediate, "w");
    if (!src || !out) {
        printf("Error opening files\n");
        return -1;
    }
    result->symtab.count = 0;

    while (fgets(line, sizeof(line), src)) {
        // remove newline, skip blank lines
        line[strlen(line)-1] = '\0';
        trim(line);
        if (line[0] == '\0' || line[0] == '.') continue;

        label[0] = opcode[0] = operand[0] = '\0';
        first[0] = rest[0] = '\0';
        // Parse line
        sscanf(line, "%s %[^\n]", first, rest);
        trim(rest);
        // Opcode or label check (1st word)
        if (lookup_optab(first) != NULL || is_directive(first)) {
            strcpy(opcode, first);
            strcpy(operand, rest);
        } else {
            strcpy(label, first);
            sscanf(rest, "%s %[^\n]", opcode, operand);
            trim(operand);
        }
        if (strcasecmp(opcode, "START") == 0) {
            loc = parse_number(operand);
            result->start_addr = loc;
            fprintf(out, "%04X\t%s\t%s\t%s\n", loc,
```

```
            label[0] ? label : "-", opcode, operand);
        continue;
    }
    if (label[0] != '\0') {
        if (add_symbol(&result->symtab, label, loc) != 0)
            return -1;
    }
    int len = 0;
    struct OptabEntry *entry = lookup_optab(opcode);

    if (entry != NULL) {
        len = entry->length;
    }
    else if (strcasecmp(opcode, "WORD") == 0) {
        len = 2;
    }
    else if (strcasecmp(opcode, "BYTE") == 0) {
        len = 1;
    }
    else if (strcasecmp(opcode, "RESB") == 0) {
        len = parse_number(operand);
    }
    else if (strcasecmp(opcode, "RESW") == 0) {
        len = 2 * parse_number(operand);
    }
    else if (strcasecmp(opcode, "END") == 0) {
        fprintf(out, "%04X\t%s\t%s\t%s\n", loc,
            label[0] ? label : "-", opcode, "-");
        break;
    }
    else {
        printf("Error: Unknown opcode %s\n", opcode);
        return -1;
    }

    fprintf(out, "%04X\t%s\t%s\t%s\n", loc, // int file
        label[0] ? label : "-", opcode, operand[0] ? operand : "-");
    loc += len;
}

fclose(src);
fclose(out);
```

```
    return 0;
}

void print_symtab(struct Symtab *symtab) {
    int i;
    printf("\nSYMTAB:\n");
    printf("Label\t\tAddress\n");
    printf("-----\t\t-------\n");
    for (i = 0; i < symtab->count; i++) {
        printf("%-10s\t%04X\n",
            symtab->entries[i].label,
            symtab->entries[i].address);
    }
}
```

Pass_2.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "assembler.h"

// register codes
int get_reg_code(char *r) {
    if (strcasecmp(r, "B") == 0) return 0;
    if (strcasecmp(r, "C") == 0) return 1;
    if (strcasecmp(r, "D") == 0) return 2;
    if (strcasecmp(r, "E") == 0) return 3;
    if (strcasecmp(r, "H") == 0) return 4;
    if (strcasecmp(r, "L") == 0) return 5;
    if (strcasecmp(r, "M") == 0) return 6;
    if (strcasecmp(r, "A") == 0) return 7;
    return -1;
}

// reg pair code: B=0, D=1, H=2, SP=3
int get_rp_code(char *rp) {
    if (strcasecmp(rp, "B") == 0) return 0;
    if (strcasecmp(rp, "D") == 0) return 1;
    if (strcasecmp(rp, "H") == 0) return 2;
    if (strcasecmp(rp, "SP") == 0) return 3;
```

```c
    return -1;
}

int parse_num(char *s) {
    int len = strlen(s);
    while (s[0] == ' ') s++;
    if (len > 0 && (s[len-1] == 'H' || s[len-1] == 'h')) {
        s[len-1] = '\0';
        return (int)strtol(s, NULL, 16);
    }
    return atoi(s);
}


int lookup_symbol(struct Symtab *tab, char *label) {
    int i;
    for (i = 0; i < tab->count; i++) {
        if (strcmp(tab->entries[i].label, label) == 0)
            return tab->entries[i].address;
    }
    return -1;
}

int get_address(char *operand, struct Symtab *symtab) {
    int addr = lookup_symbol(symtab, operand);
    if (addr >= 0) return addr;
    return parse_num(operand);
}

// ssplit "A, 05H" into reg and value
void split_operand(char *operand, char *left, char *right) {
    char *comma = strchr(operand, ',');
    if (comma) {
        strncpy(left, operand, comma - operand);
        left[comma - operand] = '\0';
        strcpy(right, comma + 1);
        // trim spaces
        while (left[0] == ' ') memmove(left, left+1, strlen(left));
        while (right[0] == ' ') memmove(right, right+1, strlen(right));
    } else {
        strcpy(left, operand);
        right[0] = '\0';
```

```
    }
}

int pass2(char *intermediate, char *output, struct Pass1Result *p1) {
    FILE *in, *out;
    char line[100];
    int loc;
    char label[20], opcode[20], operand[50];
    char objcode[30];
    char left[20], right[20];
    int op_byte, addr;
    struct OptabEntry *entry;

    in = fopen(intermediate, "r");
    out = fopen(output, "w");
    if (!in || !out) {
        printf("Error opening files\n");
        return -1;
    }

    fprintf(out, "Loc\tLabel\t\tOpcode\t\tOperand\t\tObject Code\n");
    fprintf(out, "----\t-----\t\t------\t\t-------\t\t-----------\n");

    while (fgets(line, sizeof(line), in)) {
        objcode[0] = '\0';
        sscanf(line, "%x %s %s %[^\n]", &loc, label, opcode, operand);
        if (strcasecmp(opcode, "START") == 0) {
            fprintf(out, "%04X\t%-10s\t%-10s\t%-10s\t-\n", loc,
                strcmp(label, "-") ? label : "", opcode, operand);
            continue;
        }
        if (strcasecmp(opcode, "END") == 0) {
            fprintf(out, "%04X\t%-10s\t%-10s\t%-10s\t-\n", loc,
                strcmp(label, "-") ? label : "", opcode, "");
            break;
        }
        entry = lookup_optab(opcode);

        if (entry != NULL) {
            switch (entry->kind) {
            case IK_FIXED:  // NOP, HLT
                sprintf(objcode, "%02X", entry->opcode);
```

```
                break;
            case IK_MVI:  // MVI r, data
                split_operand(operand, left, right);
                op_byte = 0x06 | (get_reg_code(left) << 3);
                sprintf(objcode, "%02X %02X", op_byte, parse_num(right) & 0xFF);
                break;
            case IK_LXI:  // LXI rp, data16
                split_operand(operand, left, right);
                op_byte = 0x01 | (get_rp_code(left) << 4);
                addr = get_address(right, &p1->symtab);
                sprintf(objcode, "%02X %02X %02X", op_byte, addr & 0xFF, (addr >> 8) &
0xFF);
                break;

            case IK_DIRECT:  // LDA, STA
                addr = get_address(operand, &p1->symtab);
                sprintf(objcode, "%02X %02X %02X", entry->opcode, addr & 0xFF, (addr >>
8) & 0xFF);
                break;
            case IK_JUMP:  // JMP, JZ, JNZ
                addr = get_address(operand, &p1->symtab);
                sprintf(objcode, "%02X %02X %02X", entry->opcode, addr & 0xFF, (addr >>
8) & 0xFF);
                break;
            case IK_MOV:  // MOV r1, r2
                split_operand(operand, left, right);
                op_byte = 0x40 | (get_reg_code(left) << 3) | get_reg_code(right);
                sprintf(objcode, "%02X", op_byte);
                break;
            case IK_REG:  // ADD, SUB
                op_byte = entry->opcode | get_reg_code(operand);
                sprintf(objcode, "%02X", op_byte);
                break;
            case IK_INRDCR:  // INR, DCR
                op_byte = entry->opcode | (get_reg_code(operand) << 3);
                sprintf(objcode, "%02X", op_byte);
                break;}
        }
        else if (strcasecmp(opcode, "WORD") == 0) {
            addr = get_address(operand, &p1->symtab);
            sprintf(objcode, "%02X %02X", addr & 0xFF, (addr >> 8) & 0xFF);
        }
```

```
        else if (strcasecmp(opcode, "BYTE") == 0) {
            sprintf(objcode, "%02X", parse_num(operand) & 0xFF);
        }
        else if (strcasecmp(opcode, "RESB") == 0 || strcasecmp(opcode, "RESW") == 0) {
            strcpy(objcode, "--");
        }
        fprintf(out, "%04X\t%-10s\t%-10s\t%-10s\t%s\n", loc,
            strcmp(label, "-") ? label : "", opcode,
            strcmp(operand, "-") ? operand : "", objcode);
    }
    fclose(in);
    fclose(out);
    return 0;
}
```

Output-1:

```
naish@MacBook-Air assembler %  ./assembler sample_data.asm && echo "---" && cat
output.obj

Running Pass 1...
Pass 1 complete.

SYMTAB:
Label           Address
-----           -------

Running Pass 2...
Assembly complete!
Output: intermediate.int, output.obj
---
Loc     Label           Opcode          Operand         Object Code
----    -----           ------          -------         -----------
0000                    MVI             A, 05H          3E 05
0002                    MVI             B, 0CH          06 0C
0004                    ADD             B               80
0005                    HLT                             76
```

Output-2:

```
naish@MacBook-Air assembler %  ./assembler sample_loop.asm && echo "---" && cat
output.obj

Running Pass 1...
Pass 1 complete.

SYMTAB:
Label           Address
-----           -------
LOOP            03ED

Running Pass 2...
Assembly complete!
Output: intermediate.int, output.obj
---
Loc     Label           Opcode          Operand         Object Code
----    -----           ------          -------         -----------
03E8                    START           1000            -
03E8                    LXI             H,1000          21 E8 03
03EB                    MVI             A,0             3E 00
03ED    LOOP            INR             A               3C
03EE                    DCR             M               35
03EF                    JNZ             LOOP            C2 ED 03
03F2                    HLT                             76
03F3                    END                             -
```